ARCHITECTURE SPEC | RELATIONAL SQL ENGINE | REACTIVE DUAL-SYNC | PHP 8.X + NODE.JS

🏛️ Smart Library Management System (SLMS)

Master System Architecture, Complete File Directory & End-to-End Workflow Specification

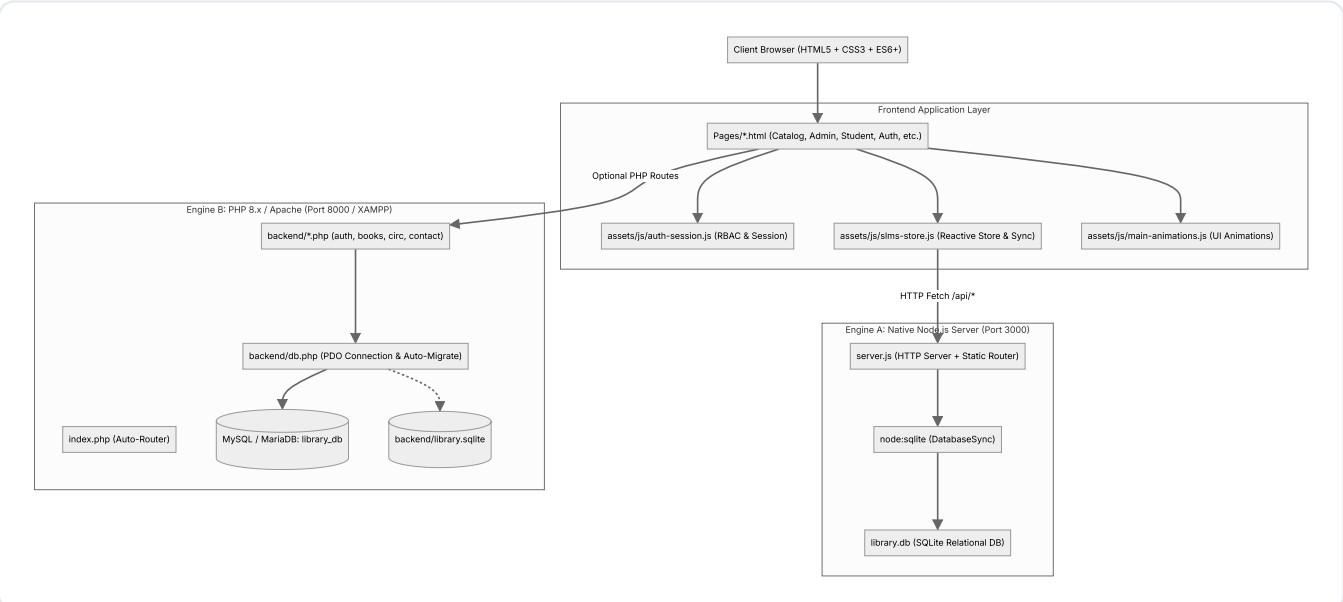
AUTHOR & DEVELOPER	APPLICATION VERSION	DATABASE ENGINE	SERVER RUNTIME
Meet Tailor (@MeetTailor-Data)	Release 1.0.0 (Production)	Relational SQLite & MySQL (PDO)	Node.js v24+ / PHP 8.x

📌 1. Executive Summary & Dual-Engine Architecture

The **Smart Library Management System (SLMS)** is a production-ready web application engineered to automate the complete lifecycle of library administration: book cataloging, member authentication, borrowing circulation, automated overdue fines calculation, and administrative inventory oversight.

The system utilizes a **Dual-Engine Model** offering maximum deployment flexibility:

- **Engine A (Primary Local Engine — Node.js + SQLite):** Runs natively via `server.js` on port `3000`. It requires zero third-party NPM packages, utilizing Node's built-in `node:sqlite` module to provide ACID-compliant relational SQL queries in `library.db`.
- **Engine B (Web / Production Engine — PHP 8.x + PDO):** Modular RESTful controllers under `backend/` supporting enterprise MySQL/MariaDB with automatic seamless fallback to a local SQLite database (`library.sqlite`).
- **Reactive State Engine (`assets/js/slms-store.js`):** Polls server endpoints every 3.5 seconds, bridges local browser storage cache with the SQL database, and broadcasts updates via window events (`slms_data_changed`).



📂 2. Exhaustive File-by-File Directory Breakdown

Detailed documentation of every file in the repository, explaining its exact functional role and capabilities.

2.1 Root Files

FILE NAME	PURPOSE & RESPONSIBILITIES	KEY ENDPOINTS & OPERATIONS
`server.js`	Zero-dependency HTTP server & relational SQLite database engine. Serves all static HTML/CSS/JS and exposes REST API endpoints.	`/api/health`, `/api/auth/login`, `/api/auth/signup`, `/api/books`, `/api/circulation/issue`, `/api/circulation/return`, `/api/circulation/all`, `/api/circulation/student/:id`, `/api/admin/overview`, `/api/users/students`, `/api/messages`.
`database.sql`	Database schema DDL script and seed dataset. Defines tables, primary keys, auto-increment, unique constraints, and foreign key cascades.	Tables: `users`, `books`, `issued_books`, `messages`. Pre-seeded with 4 default users, 10 catalog books, and 3 circulation transactions.
`index.php`	Main entry-point redirector for Apache/PHP environments.	Issues HTTP 302 redirect header to `Pages/index.html`.
`package.json`	Project manifest defining application metadata and CLI scripts.	`"scripts": { "start": "node server.js" }` for universal `npm start` execution.
`vercel.json`	Serverless platform deployment routing configuration.	Configures root routing `/` → `/Pages/index.html` and maps static assets.
`library.db`	SQLite relational binary database file managed by `server.js`.	Persists users, inventory, circulation records, and messages.

2.2 Frontend Application (`Pages/`)

PAGE FILE	FEATURES & FUNCTIONALITIES
`Pages/index.html`	**Home / Landing Page:** Displays responsive navigation, hero banner, library mission, services highlights, and a dynamically rendered 6-card featured catalog preview with real-time stock badges.
`Pages/catalog.html`	**Live Book Catalog (OPAC):** Search bar (title, author, ISBN) and category filter pills (All, Economics, Science, Technology, Literature, History). Includes interactive book modal and direct Borrow action.
`Pages/login.html`	**Authentication Center:** Dual-tab interface supporting both Student/Admin Login and new Student Registration. Features redirect retention (`?redirect=catalog.html`) and instant role routing.
`Pages/student-dashboard.html`	**Student Portal:** Personalized student space. 4 key metric cards, active borrowed books table with due date warnings and 1-click return, plus complete borrow history with fine details.

PAGE FILE	FEATURES & FUNCTIONALITIES
`Pages/admin-dashboard.html`	**Librarian Control Panel:** 5-tab interface: (1) Overview analytics & recent transactions, (2) Book Inventory CRUD & modal, (3) Circulation Issue/Return manager, (4) Registered Students directory, (5) Contact messages center.
`Pages/services.html`	**Services & FAQ:** Library facilities overview (computer lab, study rooms, archives) and interactive animated accordion FAQ answering circulation rules.
`Pages/aboutus.html`	**About Library:** Library vision, mission statement, core organizational values, and operational milestones.
`Pages/contect.html`	**Contact Us:** Physical address, hours, phone, and interactive query form submitting messages directly to the SQL database.
`Pages/blog.html`	**News & Updates Feed:** Library announcements, new book arrivals, events, and reading community articles.
`Pages/blog-post1.html`	**Blog Article View:** Full article reading view with featured photography and author credits.

2.3 Backend Engine (`backend/`)

BACKEND FILE	RESPONSIBILITIES & CONTROLLER LOGIC
`backend/config.php`	Database configuration matrix. Reads environment variables (`DB_HOST` , `DB_NAME` , etc.) or sets local defaults. Configures SQLite fallback path.
`backend/db.php`	PDO connection factory, schema migrator, automated table creator, fine computation engine (`calculate_overdue_fine()` at ₹5/day), session validators, and JSON response helper.
`backend/auth.php`	Handles `action=login` , `action=signup` , `action=logout` , and `action=get_user` . Validates unique IDs, hashes passwords, and starts PHP sessions.
`backend/books.php`	Inventory controller: `action=list` (with search/category filters), `action=get` , `action=add` (validating ISBN uniqueness), `action=edit` , and `action=delete` .
`backend/circulation.php`	Circulation controller: `action=issue` (validating student existence, max 3-book limit, duplicate issue check, and decrementing stock), `action=return` (restores stock), `action=student_borrows` , and `action=list_all` .
`backend/contact.php`	Inquiries controller: `action=submit` (saves user message to SQL) and `action=list` (loads all messages for the admin panel).

2.4 Assets (`assets/`)

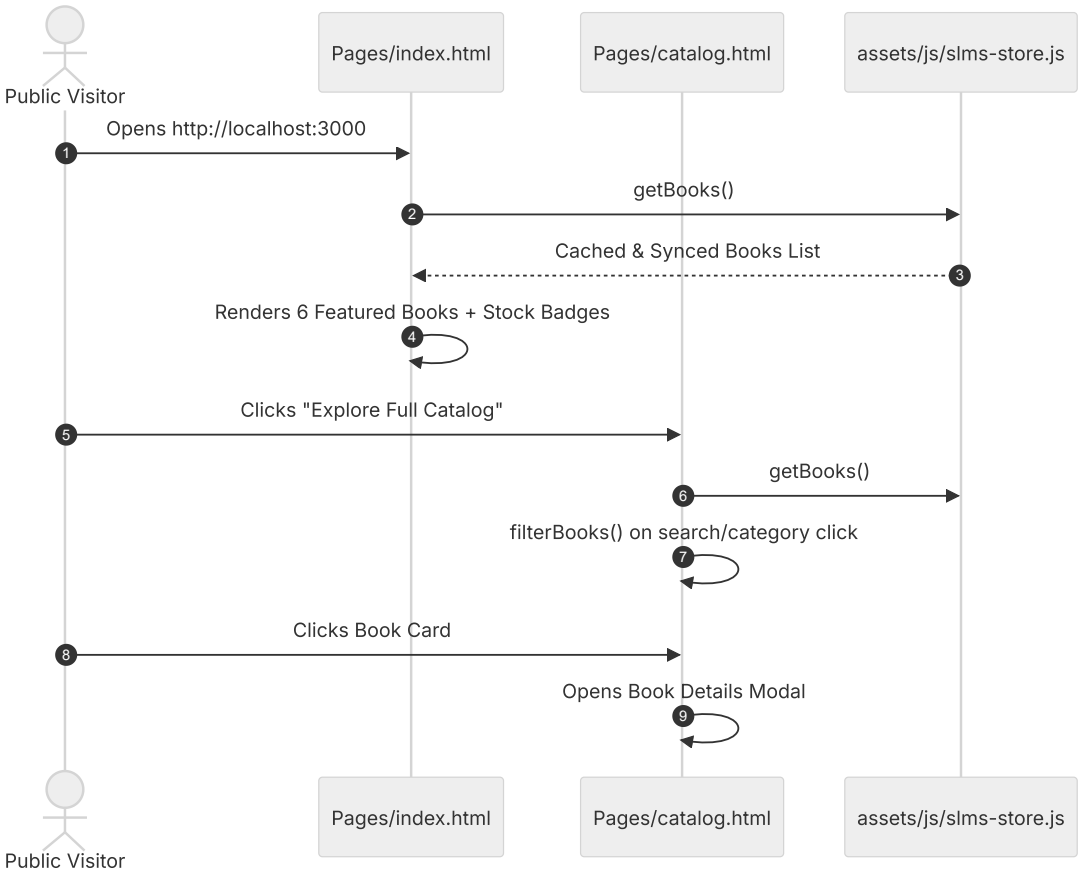
ASSET FILE	FEATURES & FUNCTIONALITIES
`assets/css/style.css`	Central design system. Defines color tokens, responsive grid layouts, card designs, badges (`badge-available` , `badge-issued` , `badge-overdue`), modal dialogues, mobile hamburger menu animations, and media queries.
`assets/js/slms-store.js`	Centralized Reactive State Engine. Automatically polls `/api/*` every 3.5s, keeps localStorage updated, handles CRUD operations, enforces 3-book limit and 14-day due date calculation, and dispatches `slms_data_changed` events.
`assets/js/auth-session.js`	Session and RBAC manager. Updates navbar buttons based on logged-in role (Admin vs Student vs Guest), protects book borrowing behind student authentication, and injects the 3-line mobile hamburger menu.
`assets/js/main-animations.js`	UI micro-animations: IntersectionObserver scroll-reveal, navbar elevation shadow on scroll, floating back-to-top button, animated number counters, and accordion FAQ toggles.

🔄 3. End-to-End User Workflows

Workflow 1: Public Discovery & Catalog Exploration (Guest Journey)

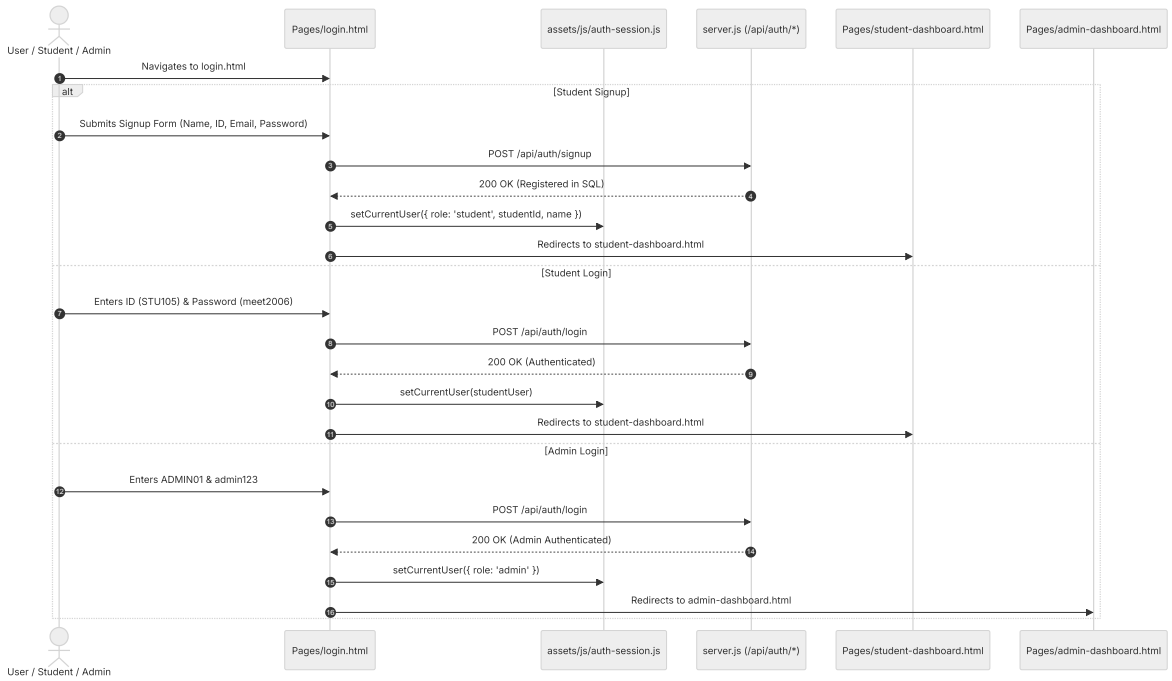
Visitors can search the full catalog, filter by subject category, and preview book details without logging in.

1. Visitor navigates to `http://localhost:3000`.
2. Home page loads and dynamically renders 6 featured catalog cards with real-time availability badges.
3. Visitor clicks "Explore Full Catalog" and is routed to `Pages/catalog.html`.
4. Visitor types keywords into the search box or clicks category buttons (Economics, Science, Technology, etc.).
5. Catalog grid dynamically re-renders in real-time matching the filter criteria.
6. Clicking a book opens the detail modal displaying cover, author, ISBN, stock, and synopsis.



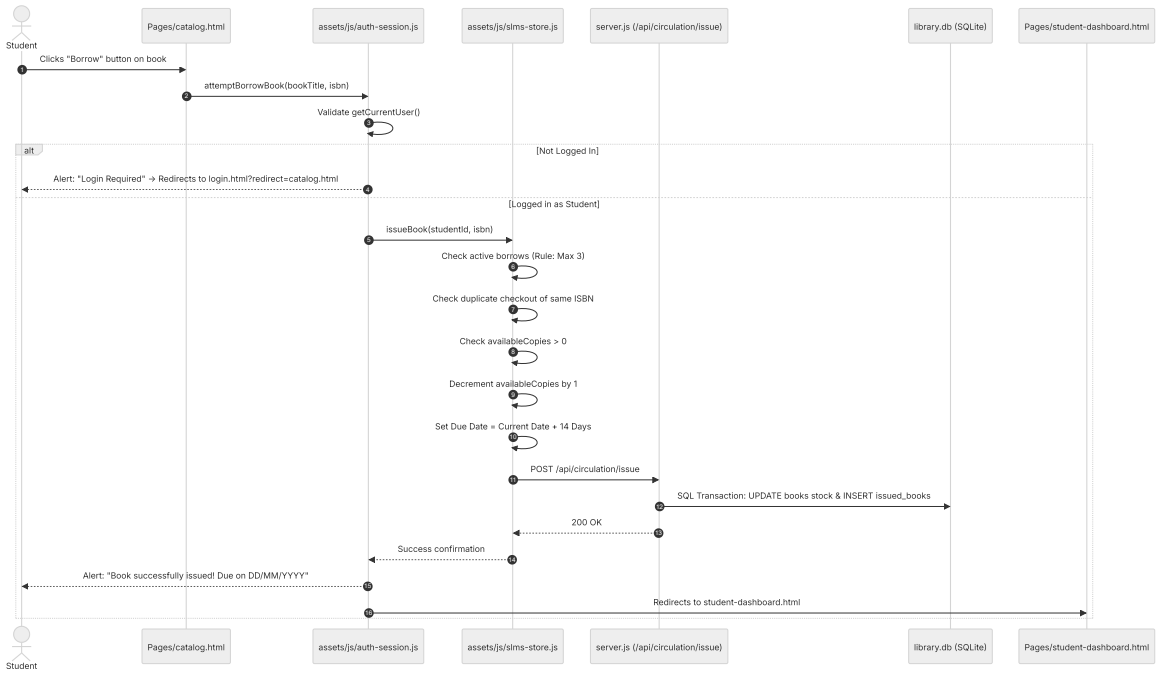
Workflow 2: Role-Based Authentication & Navigation Routing

Unified portal handling both Student and Admin credentials with auto-routing.



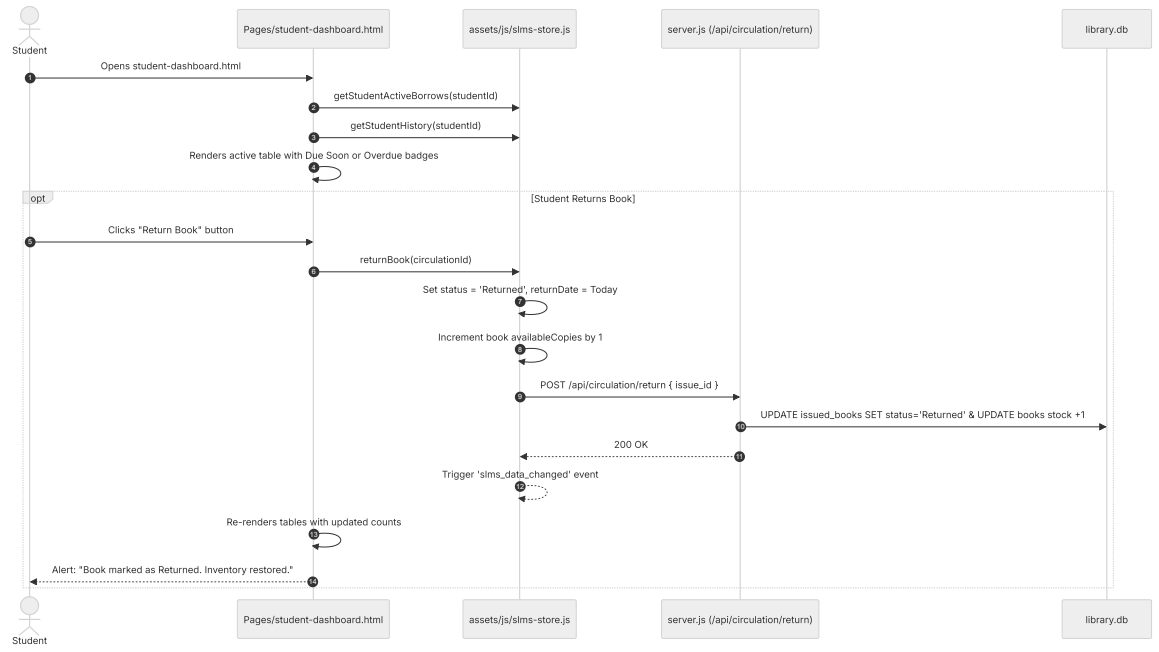
Workflow 3: Book Borrowing & Circulation Lifecycle

Enforces student login, verifies borrow limits (max 3 books), checks available stock, and calculates 14-day due dates.



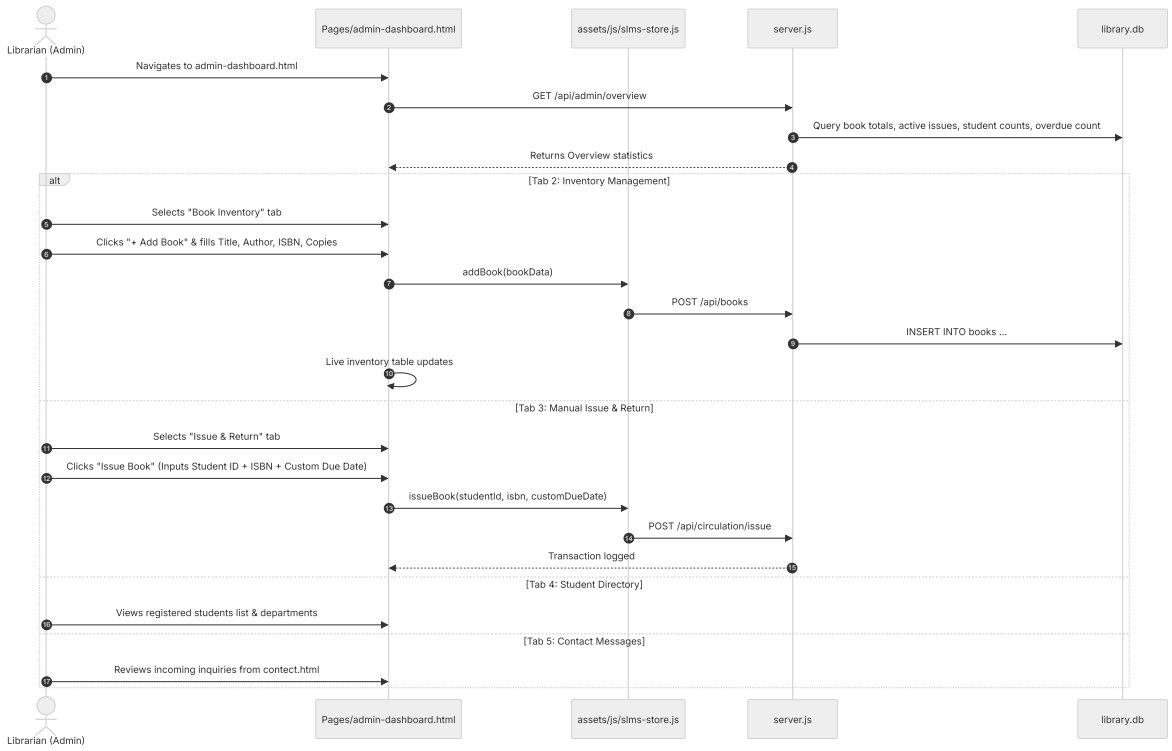
Workflow 4: Student Dashboard & Book Returns

Provides active borrow tracking, return due date countdowns, overdue penalties, and 1-click book return.



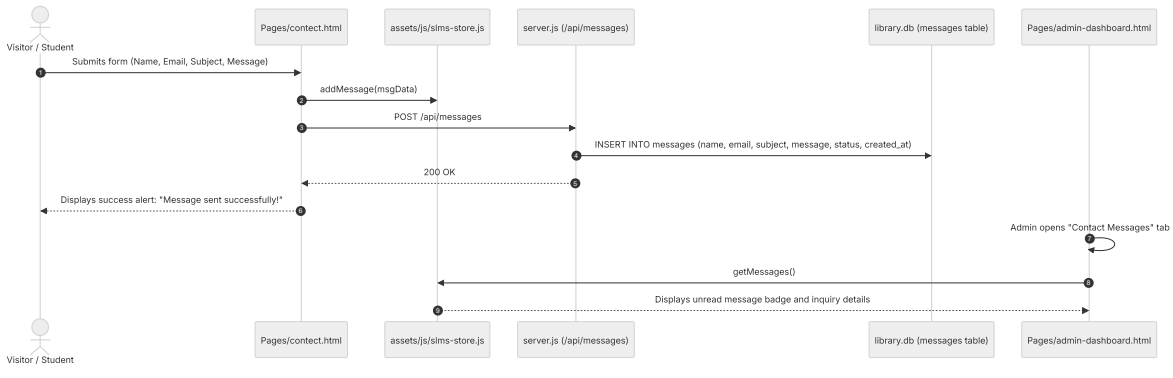
Workflow 5: Librarian Administration & Inventory Control

Complete administrative control panel for book inventory CRUD, manual issue/return, member list, and messages.



Workflow 6: Inquiries & Contact Form Workflow

Public inquiries submitted via contact page are securely stored in SQL and visible in the Admin Panel.



⚙️ 4. Business Rules & Automation Matrix

RULE NAME	BUSINESS LOGIC & CONDITIONS	ENFORCEMENT LOCATION
Default Due Date	Every issued book is automatically assigned a return deadline exactly **14 days** from checkout date (`issue_date + 14 days`).	`slms-store.js:643` `server.js:358` `circulation.php:127`
Borrow Limit	A student can hold a maximum of **3 active books** simultaneously. Additional checkout attempts are blocked with an informative limit alert.	`slms-store.js:613` `server.js:333` `circulation.php:80`
Duplicate Borrow Prevention	A student cannot check out a second copy of an ISBN they currently hold until the existing copy is returned.	`slms-store.js:619` `server.js:340` `circulation.php:91`
Real-Time Stock Sync	Issuing a book automatically decrements `available_copies` by 1. Returning increments it by 1 (capped at `total_copies`). Zero stock books are disabled.	`slms-store.js:635` `server.js:354` `circulation.php:60`
Automated Fine Engine	Calculates penalty charges of **₹5 per day** for every day a book is returned after its due date (`(return_date - due_date) * 5`).	`db.php:209` `server.js:54` `student-dashboard.html`
Authentication Gate	Guests may browse the catalog freely, but clicking "Borrow Book" prompts a login modal and redirects back after authentication.	`auth-session.js:106`
Reactive Dual-Sync	Background polling every 3.5 seconds synchronizes browser cache with SQL tables and emits `slms_data_changed` events across tabs.	`slms-store.js:382`

🚀 5. Execution & Verification Guide

> **Zero Setup Native Server:** Start the relational SQL engine instantly by running:
> `npm start` *(or* `node server.js` *)*

PORTAL / PAGE	LOCAL URL	CREDENTIALS & ROLE
Public Home	`http://localhost:3000`	Guest / Public
Book Catalog	`http://localhost:3000/Pages/catalog.html`	Guest & Student
Login & Signup	`http://localhost:3000/Pages/login.html`	All Users
Admin Dashboard	`http://localhost:3000/Pages/admin-dashboard.html`	`ADMIN01` / `admin123`
Student Dashboard	`http://localhost:3000/Pages/student-dashboard.html`	`STU105` / `meet2006` *(or* `STU101` / `student123` *)*